

StreetRace Manager Integration Testing Report

Total integration test cases: 154

Passed: 154

Failed: 0

1. Integration Scope And Verified State

StreetRace Manager was audited both at the module level and at the cross-module level. The repository contains the required modules Registration, Crew Management, Inventory, Race Management, Results, and Mission Planning, plus the additional Garage and Leaderboard modules.

The audited integration suite was re-run locally and all 154 collected cases passed.

2. Cross-Module Focus

The integration work checks role assignment before racing, car-condition rules, shared availability state, mission staffing, prize-money flow into inventory, driver statistics, ranking updates, garage repair effects, and conflicts between missions and races.

This report now explains every collected case instead of only the top-level scenarios.

3. Complete Integration Case Catalog

test_crew_management.py - TestAssignRole (crew_management)

Case I-001:

integration/tests/test_crew_management.py::TestAssignRole::test_assign_role_valid_to_registered_membe

r

Scenario: Assign role valid to registered member.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the crew_management module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-002:

integration/tests/test_crew_management.py::TestAssignRole::test_assign_role_to_unregistered_member

Scenario: Assign role to unregistered member.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the crew_management module.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-003: integration/tests/test_crew_management.py::TestAssignRole::test_assign_role_invalid_role

Scenario: Assign role invalid role.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the crew_management module.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-004:

integration/tests/test_crew_management.py::TestAssignRole::test_assign_role_all_valid_transitions

Scenario: Assign role all valid transitions.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the crew_management module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

test_crew_management.py - TestUpdateSkillLevel (crew_management)

Case I-005:

integration/tests/test_crew_management.py::TestUpdateSkillLevel::test_update_skill_level_valid

Scenario: Update skill level valid.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

crew_management module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-006:

integration/tests/test_crew_management.py::TestUpdateSkillLevel::test_update_skill_level_boundary_1

Scenario: Update skill level boundary 1.

Why Needed: This case is needed because crew_management must handle boundary and invalid

numeric inputs safely.

Expected Result: The boundary value should be accepted and handled correctly.

Observed Result: Passed locally in the audited integration suite.

Case I-007:

integration/tests/test_crew_management.py::TestUpdateSkillLevel::test_update_skill_level_boundary_10

Scenario: Update skill level boundary 10.

Why Needed: This case is needed because crew_management must handle boundary and invalid

numeric inputs safely.

Expected Result: The boundary value should be accepted and handled correctly.

Observed Result: Passed locally in the audited integration suite.

Case I-008:

integration/tests/test_crew_management.py::TestUpdateSkillLevel::test_update_skill_level_zero_raises_err

or

Scenario: Update skill level zero raises error.

Why Needed: This case is needed because crew_management must handle boundary and invalid

numeric inputs safely.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-009:

integration/tests/test_crew_management.py::TestUpdateSkillLevel::test_update_skill_level_11_raises_error

Scenario: Update skill level 11 raises error.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

crew_management module.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-010:

integration/tests/test_crew_management.py::TestUpdateSkillLevel::test_update_skill_level_nonexistent_member

Scenario: Update skill level nonexistent member.

Why Needed: This case is needed because crew_management must reject stale or invalid identifiers cleanly.

Expected Result: The operation should fail with KeyError because the referenced record does not exist.

Observed Result: Passed locally in the audited integration suite.

test_crew_management.py - TestSetAvailability (crew_management)

Case I-011: integration/tests/test_crew_management.py::TestSetAvailability::test_set_available_true

Scenario: Set available true.

Why Needed: This case is needed because availability state is shared across modules and can easily become inconsistent.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-012: integration/tests/test_crew_management.py::TestSetAvailability::test_set_available_false

Scenario: Set available false.

Why Needed: This case is needed because availability state is shared across modules and can easily become inconsistent.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-013:

integration/tests/test_crew_management.py::TestSetAvailability::test_set_availability_nonexistent_member

Scenario: Set availability nonexistent member.

Why Needed: This case is needed because crew_management must reject stale or invalid identifiers

cleanly.

Expected Result: The operation should fail with KeyError because the referenced record does not exist.

Observed Result: Passed locally in the audited integration suite.

test_crew_management.py - TestGetAvailableByRole (crew_management)

Case I-014:

integration/tests/test_crew_management.py::TestGetAvailableByRole::test_get_available_by_role_empty

Scenario: Get available by role empty.

Why Needed: This case is needed because availability state is shared across modules and can easily

become inconsistent.

Expected Result: The empty-state query should return an empty but well-formed result.

Observed Result: Passed locally in the audited integration suite.

Case I-015:

integration/tests/test_crew_management.py::TestGetAvailableByRole::test_get_available_by_role_returns_
only_available

Scenario: Get available by role returns only available.

Why Needed: This case is needed because availability state is shared across modules and can easily
become inconsistent.

Expected Result: The query should return only the matching records in the correct order.

Observed Result: Passed locally in the audited integration suite.

Case I-016:

integration/tests/test_crew_management.py::TestGetAvailableByRole::test_get_available_by_role_returns_
only_matching_role

Scenario: Get available by role returns only matching role.

Why Needed: This case is needed because availability state is shared across modules and can easily
become inconsistent.

Expected Result: The query should return only the matching records in the correct order.

Observed Result: Passed locally in the audited integration suite.

Case I-017:

integration/tests/test_crew_management.py::TestGetAvailableByRole::test_get_available_by_role_multiple_
available

Scenario: Get available by role multiple available.

Why Needed: This case is needed because availability state is shared across modules and can easily become inconsistent.

Expected Result: The lookup should return the requested record set in a valid structure.

Observed Result: Passed locally in the audited integration suite.

test_garage.py - TestRepairCar (garage)

Case I-018: integration/tests/test_garage.py::TestRepairCar::test_repair_car_valid_mechanic

Scenario: Repair car valid mechanic.

Why Needed: This case is needed because vehicle-condition rules affect how garage interacts with race and mission flows.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-019: integration/tests/test_garage.py::TestRepairCar::test_repair_car_non_mechanic_fails

Scenario: Repair car non mechanic fails.

Why Needed: This case is needed because vehicle-condition rules affect how garage interacts with race and mission flows.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-020: integration/tests/test_garage.py::TestRepairCar::test_repair_car_unavailable_mechanic_fails

Scenario: Repair car unavailable mechanic fails.

Why Needed: This case is needed because availability state is shared across modules and can easily become inconsistent.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-021: integration/tests/test_garage.py::TestRepairCar::test_repair_car_non_damaged_fails

Scenario: Repair car non damaged fails.

Why Needed: This case is needed because vehicle-condition rules affect how garage interacts with race and mission flows.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-022: integration/tests/test_garage.py::TestRepairCar::test_repair_car_nonexistent_car_fails

Scenario: Repair car nonexistent car fails.

Why Needed: This case is needed because garage must reject stale or invalid identifiers cleanly.

Expected Result: The operation should fail with KeyError because the referenced record does not exist.

Observed Result: Passed locally in the audited integration suite.

Case I-023: integration/tests/test_garage.py::TestRepairCar::test_repair_car_nonexistent_mechanic_fails

Scenario: Repair car nonexistent mechanic fails.

Why Needed: This case is needed because garage must reject stale or invalid identifiers cleanly.

Expected Result: The operation should fail with KeyError because the referenced record does not exist.

Observed Result: Passed locally in the audited integration suite.

Case I-024: integration/tests/test_garage.py::TestRepairCar::test_repair_car_adds_to_log

Scenario: Repair car adds to log.

Why Needed: This case is needed because vehicle-condition rules affect how garage interacts with race

and mission flows.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

test_garage.py - TestGetRepairLog (garage)

Case I-025: integration/tests/test_garage.py::TestGetRepairLog::test_get_repair_log_empty

Scenario: Get repair log empty.

Why Needed: This case is needed because vehicle-condition rules affect how garage interacts with race

and mission flows.

Expected Result: The empty-state query should return an empty but well-formed result.

Observed Result: Passed locally in the audited integration suite.

Case I-026: integration/tests/test_garage.py::TestGetRepairLog::test_get_repair_log_multiple_repairs

Scenario: Get repair log multiple repairs.

Why Needed: This case is needed because vehicle-condition rules affect how garage interacts with race and mission flows.

Expected Result: The lookup should return the requested record set in a valid structure.

Observed Result: Passed locally in the audited integration suite.

test_garage.py - TestGetCarsNeedingRepair (garage)

Case I-027:

integration/tests/test_garage.py::TestGetCarsNeedingRepair::test_get_cars_needing_repair_empty

Scenario: Get cars needing repair empty.

Why Needed: This case is needed because vehicle-condition rules affect how garage interacts with race and mission flows.

Expected Result: The empty-state query should return an empty but well-formed result.

Observed Result: Passed locally in the audited integration suite.

Case I-028:

integration/tests/test_garage.py::TestGetCarsNeedingRepair::test_get_cars_needing_repair_returns_damaged_only

Scenario: Get cars needing repair returns damaged only.

Why Needed: This case is needed because vehicle-condition rules affect how garage interacts with race and mission flows.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-029:

integration/tests/test_garage.py::TestGetCarsNeedingRepair::test_get_cars_needing_repair_multiple_damaged

Scenario: Get cars needing repair multiple damaged.

Why Needed: This case is needed because vehicle-condition rules affect how garage interacts with race and mission flows.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

test_integration.py - TestIntegrationScenarios (cross-module integration)

Case I-030:

integration/tests/test_integration.py::TestIntegrationScenarios::test_scenario_1_full_race_pipeline_updates_cash

Scenario: Scenario 1 full race pipeline updates cash.

Why Needed: This case is needed because it checks a real cross-module workflow rather than a single isolated function call.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-031:

integration/tests/test_integration.py::TestIntegrationScenarios::test_scenario_2_non_driver_cannot_enter_race

Scenario: Scenario 2 non driver cannot enter race.

Why Needed: This case is needed because it checks a real cross-module workflow rather than a single isolated function call.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-032:

integration/tests/test_integration.py::TestIntegrationScenarios::test_scenario_3_unregistered_member_cannot_enter_race

Scenario: Scenario 3 unregistered member cannot enter race.

Why Needed: This case is needed because it checks a real cross-module workflow rather than a single isolated function call.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-033:

integration/tests/test_integration.py::TestIntegrationScenarios::test_scenario_4_damaged_car_cannot_be_assigned_to_race

Scenario: Scenario 4 damaged car cannot be assigned to race.

Why Needed: This case is needed because it checks a real cross-module workflow rather than a single isolated function call.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-034:

integration/tests/test_integration.py::TestIntegrationScenarios::test_scenario_5_mission_fails_when_no_available_driver

Scenario: Scenario 5 mission fails when no available driver.

Why Needed: This case is needed because it checks a real cross-module workflow rather than a single isolated function call.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-035:

integration/tests/test_integration.py::TestIntegrationScenarios::test_scenario_6_mission_with_damaged_car_requires_mechanic

Scenario: Scenario 6 mission with damaged car requires mechanic.

Why Needed: This case is needed because it checks a real cross-module workflow rather than a single isolated function call.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-036:

integration/tests/test_integration.py::TestIntegrationScenarios::test_scenario_7_mission_succeeds_with_all
_roles_available

Scenario: Scenario 7 mission succeeds with all roles available.

Why Needed: This case is needed because it checks a real cross-module workflow rather than a single
isolated function call.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-037:

integration/tests/test_integration.py::TestIntegrationScenarios::test_scenario_8_driver_available_after_race
_completion

Scenario: Scenario 8 driver available after race completion.

Why Needed: This case is needed because it checks a real cross-module workflow rather than a single
isolated function call.

Expected Result: Completing the workflow should release the reserved resource back into availability.

Observed Result: Passed locally in the audited integration suite.

Case I-038:

integration/tests/test_integration.py::TestIntegrationScenarios::test_scenario_9_leaderboard_reflects_race_outcomes

Scenario: Scenario 9 leaderboard reflects race outcomes.

Why Needed: This case is needed because it checks a real cross-module workflow rather than a single isolated function call.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-039:

integration/tests/test_integration.py::TestIntegrationScenarios::test_scenario_10_repaired_car_can_be_assigned_to_race

Scenario: Scenario 10 repaired car can be assigned to race.

Why Needed: This case is needed because it checks a real cross-module workflow rather than a single isolated function call.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-040:

integration/tests/test_integration.py::TestIntegrationScenarios::test_scenario_11_role_change_enables_race_entry

Scenario: Scenario 11 role change enables race entry.

Why Needed: This case is needed because it checks a real cross-module workflow rather than a single isolated function call.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-041:

integration/tests/test_integration.py::TestIntegrationScenarios::test_scenario_12_complete_mission_frees_a
ll_assigned_crew

Scenario: Scenario 12 complete mission frees all assigned crew.

Why Needed: This case is needed because it checks a real cross-module workflow rather than a single isolated function call.

Expected Result: Completing the workflow should release the reserved resource back into availability.

Observed Result: Passed locally in the audited integration suite.

test_integration.py - TestCrossModuleInteractions (cross-module integration)

Case I-042:

integration/tests/test_integration.py::TestCrossModuleInteractions::test_sequential_races_with_multiple_driv
ers

Scenario: Sequential races with multiple drivers.

Why Needed: This case is needed because it checks a real cross-module workflow rather than a single isolated function call.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-043:

integration/tests/test_integration.py::TestCrossModuleInteractions::test_mission_concurrent_with_races

Scenario: Mission concurrent with races.

Why Needed: This case is needed because it checks a real cross-module workflow rather than a single isolated function call.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

test_inventory.py - TestAddCar (inventory)

Case I-044: integration/tests/test_inventory.py::TestAddCar::test_add_car_valid

Scenario: Add car valid.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the inventory module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-045: integration/tests/test_inventory.py::TestAddCar::test_add_car_speed_rating_boundary_1

Scenario: Add car speed rating boundary 1.

Why Needed: This case is needed because inventory must handle boundary and invalid numeric inputs

safely.

Expected Result: The boundary value should be accepted and handled correctly.

Observed Result: Passed locally in the audited integration suite.

Case I-046: integration/tests/test_inventory.py::TestAddCar::test_add_car_speed_rating_boundary_10

Scenario: Add car speed rating boundary 10.

Why Needed: This case is needed because inventory must handle boundary and invalid numeric inputs

safely.

Expected Result: The boundary value should be accepted and handled correctly.

Observed Result: Passed locally in the audited integration suite.

Case I-047:

integration/tests/test_inventory.py::TestAddCar::test_add_car_speed_rating_zero_raises_error

Scenario: Add car speed rating zero raises error.

Why Needed: This case is needed because inventory must handle boundary and invalid numeric inputs

safely.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-048: integration/tests/test_inventory.py::TestAddCar::test_add_car_speed_rating_11_raises_error

Scenario: Add car speed rating 11 raises error.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the inventory module.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

test_inventory.py - TestGetCar (inventory)

Case I-049: integration/tests/test_inventory.py::TestGetCar::test_get_car_valid_id

Scenario: Get car valid id.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the inventory module.

Expected Result: The lookup should return the requested record set in a valid structure.

Observed Result: Passed locally in the audited integration suite.

Case I-050: integration/tests/test_inventory.py::TestGetCar::test_get_car_invalid_id_raises_keyerror

Scenario: Get car invalid id raises keyerror.

Why Needed: This case is needed because inventory must reject stale or invalid identifiers cleanly.

Expected Result: The operation should fail with KeyError because the referenced record does not exist.

Observed Result: Passed locally in the audited integration suite.

test_inventory.py - TestListCars (inventory)

Case I-051: integration/tests/test_inventory.py::TestListCars::test_list_cars_empty

Scenario: List cars empty.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the inventory module.

Expected Result: The empty-state query should return an empty but well-formed result.

Observed Result: Passed locally in the audited integration suite.

Case I-052: integration/tests/test_inventory.py::TestListCars::test_list_cars_returns_all

Scenario: List cars returns all.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the inventory module.

Expected Result: The lookup should return the requested record set in a valid structure.

Observed Result: Passed locally in the audited integration suite.

Case I-053: integration/tests/test_inventory.py::TestListCars::test_list_cars_available_only_filter

Scenario: List cars available only filter.

Why Needed: This case is needed because availability state is shared across modules and can easily become inconsistent.

Expected Result: The query should return only the matching records in the correct order.

Observed Result: Passed locally in the audited integration suite.

test_inventory.py - TestUpdateCarCondition (inventory)

Case I-054:

integration/tests/test_inventory.py::TestUpdateCarCondition::test_update_car_condition_excellent

Scenario: Update car condition excellent.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

inventory module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-055: integration/tests/test_inventory.py::TestUpdateCarCondition::test_update_car_condition_good

Scenario: Update car condition good.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

inventory module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-056:

integration/tests/test_inventory.py::TestUpdateCarCondition::test_update_car_condition_damaged

Scenario: Update car condition damaged.

Why Needed: This case is needed because vehicle-condition rules affect how inventory interacts with race and mission flows.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-057:

integration/tests/test_inventory.py::TestUpdateCarCondition::test_update_car_condition_invalid_raises_error

r

Scenario: Update car condition invalid raises error.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the inventory module.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-058:

integration/tests/test_inventory.py::TestUpdateCarCondition::test_update_car_condition_nonexistent_car

Scenario: Update car condition nonexistent car.

Why Needed: This case is needed because inventory must reject stale or invalid identifiers cleanly.

Expected Result: The operation should fail with KeyError because the referenced record does not exist.

Observed Result: Passed locally in the audited integration suite.

test_inventory.py - TestAddInventoryItem (inventory)

Case I-059: integration/tests/test_inventory.py::TestAddInventoryItem::test_add_inventory_item_part_valid

Scenario: Add inventory item part valid.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the inventory module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-060: integration/tests/test_inventory.py::TestAddInventoryItem::test_add_inventory_item_tool_valid

Scenario: Add inventory item tool valid.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the inventory module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-061:

integration/tests/test_inventory.py::TestAddInventoryItem::test_add_inventory_item_invalid_type

Scenario: Add inventory item invalid type.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the inventory module.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-062:

integration/tests/test_inventory.py::TestAddInventoryItem::test_add_inventory_item_zero_quantity_raises_er

ror

Scenario: Add inventory item zero quantity raises error.

Why Needed: This case is needed because inventory must handle boundary and invalid numeric inputs

safely.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-063:

integration/tests/test_inventory.py::TestAddInventoryItem::test_add_inventory_item_negative_quantity_raise

s_error

Scenario: Add inventory item negative quantity raises error.

Why Needed: This case is needed because inventory must handle boundary and invalid numeric inputs

safely.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

test_inventory.py - TestCashBalance (inventory)

Case I-064: integration/tests/test_inventory.py::TestCashBalance::test_get_cash_balance_initial

Scenario: Get cash balance initial.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

inventory module.

Expected Result: The lookup should return the requested record set in a valid structure.

Observed Result: Passed locally in the audited integration suite.

Case I-065: integration/tests/test_inventory.py::TestCashBalance::test_add_cash_positive_amount

Scenario: Add cash positive amount.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

inventory module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-066: integration/tests/test_inventory.py::TestCashBalance::test_add_cash_zero_raises_error

Scenario: Add cash zero raises error.

Why Needed: This case is needed because inventory must handle boundary and invalid numeric inputs

safely.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-067: integration/tests/test_inventory.py::TestCashBalance::test_add_cash_negative_raises_error

Scenario: Add cash negative raises error.

Why Needed: This case is needed because inventory must handle boundary and invalid numeric inputs

safely.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-068: integration/tests/test_inventory.py::TestCashBalance::test_deduct_cash_valid_amount

Scenario: Deduct cash valid amount.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

inventory module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-069: integration/tests/test_inventory.py::TestCashBalance::test_deduct_cash_entire_balance

Scenario: Deduct cash entire balance.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

inventory module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-070:

integration/tests/test_inventory.py::TestCashBalance::test_deduct_cash_more_than_balance_raises_error

Scenario: Deduct cash more than balance raises error.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the inventory module.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-071: integration/tests/test_inventory.py::TestCashBalance::test_deduct_cash_zero_raises_error

Scenario: Deduct cash zero raises error.

Why Needed: This case is needed because inventory must handle boundary and invalid numeric inputs safely.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-072:

integration/tests/test_inventory.py::TestCashBalance::test_deduct_cash_negative_raises_error

Scenario: Deduct cash negative raises error.

Why Needed: This case is needed because inventory must handle boundary and invalid numeric inputs safely.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-073: integration/tests/test_inventory.py::TestCashBalance::test_add_and_deduct_sequence

Scenario: Add and deduct sequence.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the inventory module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

test_leaderboard.py - TestGetFullLeaderboard (leaderboard)

Case I-074:

integration/tests/test_leaderboard.py::TestGetFullLeaderboard::test_get_full_leaderboard_empty

Scenario: Get full leaderboard empty.

Why Needed: This case is needed because downstream views depend on leaderboard returning correctly ordered data.

Expected Result: The empty-state query should return an empty but well-formed result.

Observed Result: Passed locally in the audited integration suite.

Case I-075:

integration/tests/test_leaderboard.py::TestGetFullLeaderboard::test_get_full_leaderboard_single_entry

Scenario: Get full leaderboard single entry.

Why Needed: This case is needed because downstream views depend on leaderboard returning correctly

ordered data.

Expected Result: The lookup should return the requested record set in a valid structure.

Observed Result: Passed locally in the audited integration suite.

Case I-076:

integration/tests/test_leaderboard.py::TestGetFullLeaderboard::test_get_full_leaderboard_multiple_entries_

sorted

Scenario: Get full leaderboard multiple entries sorted.

Why Needed: This case is needed because downstream views depend on leaderboard returning correctly

ordered data.

Expected Result: The query should return only the matching records in the correct order.

Observed Result: Passed locally in the audited integration suite.

test_leaderboard.py - TestGetTopDriver (leaderboard)

Case I-077: integration/tests/test_leaderboard.py::TestGetTopDriver::test_get_top_driver_empty

Scenario: Get top driver empty.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

leaderboard module.

Expected Result: The empty-state query should return an empty but well-formed result.

Observed Result: Passed locally in the audited integration suite.

Case I-078: integration/tests/test_leaderboard.py::TestGetTopDriver::test_get_top_driver_three_drivers

Scenario: Get top driver three drivers.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

leaderboard module.

Expected Result: The lookup should return the requested record set in a valid structure.

Observed Result: Passed locally in the audited integration suite.

test_leaderboard.py - TestGetMissionSuccessRate (leaderboard)

Case I-079:

integration/tests/test_leaderboard.py::TestGetMissionSuccessRate::test_get_mission_success_rate_no_mis

sions

Scenario: Get mission success rate no missions.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

leaderboard module.

Expected Result: The lookup should return the requested record set in a valid structure.

Observed Result: Passed locally in the audited integration suite.

Case I-080:

integration/tests/test_leaderboard.py::TestGetMissionSuccessRate::test_get_mission_success_rate_all_co

mpleted

Scenario: Get mission success rate all completed.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

leaderboard module.

Expected Result: The lookup should return the requested record set in a valid structure.

Observed Result: Passed locally in the audited integration suite.

Case I-081:

integration/tests/test_leaderboard.py::TestGetMissionSuccessRate::test_get_mission_success_rate_mixed

Scenario: Get mission success rate mixed.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

leaderboard module.

Expected Result: The lookup should return the requested record set in a valid structure.

Observed Result: Passed locally in the audited integration suite.

Case I-082:

integration/tests/test_leaderboard.py::TestGetMissionSuccessRate::test_get_mission_success_rate_all_fail

ed

Scenario: Get mission success rate all failed.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

leaderboard module.

Expected Result: The lookup should return the requested record set in a valid structure.

Observed Result: Passed locally in the audited integration suite.

test_mission_planning.py - TestCreateMission (mission_planning)

Case I-083:

integration/tests/test_mission_planning.py::TestCreateMission::test_create_mission_delivery_valid

Scenario: Create mission delivery valid.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the mission_planning module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-084:

integration/tests/test_mission_planning.py::TestCreateMission::test_create_mission_rescue_valid

Scenario: Create mission rescue valid.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the mission_planning module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-085:

integration/tests/test_mission_planning.py::TestCreateMission::test_create_mission_surveillance_valid

Scenario: Create mission surveillance valid.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

mission_planning module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-086:

integration/tests/test_mission_planning.py::TestCreateMission::test_create_mission_invalid_type_fails

Scenario: Create mission invalid type fails.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

mission_planning module.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-087:

integration/tests/test_mission_planning.py::TestCreateMission::test_create_mission_empty_required_roles_

fails

Scenario: Create mission empty required roles fails.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

mission_planning module.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-088:

integration/tests/test_mission_planning.py::TestCreateMission::test_create_mission_invalid_role_in_require
d_fails

Scenario: Create mission invalid role in required fails.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the
mission_planning module.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-089:

integration/tests/test_mission_planning.py::TestCreateMission::test_create_mission_with_damaged_car

Scenario: Create mission with damaged car.

Why Needed: This case is needed because vehicle-condition rules affect how mission_planning interacts
with race and mission flows.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-090:

integration/tests/test_mission_planning.py::TestCreateMission::test_create_mission_without_damaged_car

Scenario: Create mission without damaged car.

Why Needed: This case is needed because vehicle-condition rules affect how mission_planning interacts

with race and mission flows.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

test_mission_planning.py - TestAssignMission (mission_planning)

Case I-091:

integration/tests/test_mission_planning.py::TestAssignMission::test_assign_mission_single_role_success

Scenario: Assign mission single role success.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

mission_planning module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-092:

integration/tests/test_mission_planning.py::TestAssignMission::test_assign_mission_multiple_roles_success

Scenario: Assign mission multiple roles success.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

mission_planning module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-093:

integration/tests/test_mission_planning.py::TestAssignMission::test_assign_mission_no_available_driver_fai

ls

Scenario: Assign mission no available driver fails.

Why Needed: This case is needed because availability state is shared across modules and can easily become inconsistent.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-094:

integration/tests/test_mission_planning.py::TestAssignMission::test_assign_mission_no_available_mechani
c_fails

Scenario: Assign mission no available mechanic fails.

Why Needed: This case is needed because availability state is shared across modules and can easily become inconsistent.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-095:

integration/tests/test_mission_planning.py::TestAssignMission::test_assign_mission_marks_unavailable

Scenario: Assign mission marks unavailable.

Why Needed: This case is needed because availability state is shared across modules and can easily become inconsistent.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-096:

integration/tests/test_mission_planning.py::TestAssignMission::test_assign_mission_does_not_duplicate_members

Scenario: Assign mission does not duplicate members.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the mission_planning module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

test_mission_planning.py - TestCompleteMission (mission_planning)

Case I-097:

integration/tests/test_mission_planning.py::TestCompleteMission::test_complete_mission_marks_status_completed

Scenario: Complete mission marks status completed.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the mission_planning module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-098:

integration/tests/test_mission_planning.py::TestCompleteMission::test_complete_mission_frees_assigned_c

rew

Scenario: Complete mission frees assigned crew.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

mission_planning module.

Expected Result: Completing the workflow should release the reserved resource back into availability.

Observed Result: Passed locally in the audited integration suite.

Case I-099:

integration/tests/test_mission_planning.py::TestCompleteMission::test_complete_mission_frees_multiple_cr

ew

Scenario: Complete mission frees multiple crew.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

mission_planning module.

Expected Result: Completing the workflow should release the reserved resource back into availability.

Observed Result: Passed locally in the audited integration suite.

test_mission_planning.py - TestFailMission (mission_planning)

Case I-100:

integration/tests/test_mission_planning.py::TestFailMission::test_fail_mission_marks_status_failed

Scenario: Fail mission marks status failed.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the mission_planning module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-101:

integration/tests/test_mission_planning.py::TestFailMission::test_fail_mission_frees_assigned_crew

Scenario: Fail mission frees assigned crew.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the mission_planning module.

Expected Result: Completing the workflow should release the reserved resource back into availability.

Observed Result: Passed locally in the audited integration suite.

test_mission_planning.py - TestGetMission (mission_planning)

Case I-102: integration/tests/test_mission_planning.py::TestGetMission::test_get_mission_valid_id

Scenario: Get mission valid id.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the mission_planning module.

Expected Result: The lookup should return the requested record set in a valid structure.

Observed Result: Passed locally in the audited integration suite.

Case I-103:

integration/tests/test_mission_planning.py::TestGetMission::test_get_mission_invalid_id_raises_keyerror

Scenario: Get mission invalid id raises keyerror.

Why Needed: This case is needed because mission_planning must reject stale or invalid identifiers cleanly.

Expected Result: The operation should fail with KeyError because the referenced record does not exist.

Observed Result: Passed locally in the audited integration suite.

test_mission_planning.py - TestMissionIntegrationRules (mission_planning)

Case I-104:

integration/tests/test_mission_planning.py::TestMissionIntegrationRules::test_mission_with_damaged_car_requires_available_mechanic

Scenario: Mission with damaged car requires available mechanic.

Why Needed: This case is needed because availability state is shared across modules and can easily become inconsistent.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-105:

integration/tests/test_mission_planning.py::TestMissionIntegrationRules::test_mission_role_reassignment_n
ot_same_person

Scenario: Mission role reassignment not same person.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the
mission_planning module.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

test_race_management.py - TestCreateRace (race_management)

Case I-106: integration/tests/test_race_management.py::TestCreateRace::test_create_race_valid

Scenario: Create race valid.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the
race_management module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-107:

integration/tests/test_race_management.py::TestCreateRace::test_create_race_zero_prize_raises_error

Scenario: Create race zero prize raises error.

Why Needed: This case is needed because race_management must handle boundary and invalid

numeric inputs safely.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-108:

integration/tests/test_race_management.py::TestCreateRace::test_create_race_negative_prize_raises_error

r

Scenario: Create race negative prize raises error.

Why Needed: This case is needed because race_management must handle boundary and invalid

numeric inputs safely.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

test_race_management.py - TestAssignDriverToRace (race_management)

Case I-109:

integration/tests/test_race_management.py::TestAssignDriverToRace::test_assign_driver_valid_driver

Scenario: Assign driver valid driver.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

race_management module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-110:

integration/tests/test_race_management.py::TestAssignDriverToRace::test_assign_driver_mechanic_fails

Scenario: Assign driver mechanic fails.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the race_management module.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-111:

integration/tests/test_race_management.py::TestAssignDriverToRace::test_assign_driver_unavailable_fails

Scenario: Assign driver unavailable fails.

Why Needed: This case is needed because availability state is shared across modules and can easily become inconsistent.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-112:

integration/tests/test_race_management.py::TestAssignDriverToRace::test_assign_driver_nonexistent_fails

Scenario: Assign driver nonexistent fails.

Why Needed: This case is needed because race_management must reject stale or invalid identifiers cleanly.

Expected Result: The operation should fail with KeyError because the referenced record does not exist.

Observed Result: Passed locally in the audited integration suite.

Case I-113:

integration/tests/test_race_management.py::TestAssignDriverToRace::test_assign_driver_sets_unavailable

Scenario: Assign driver sets unavailable.

Why Needed: This case is needed because availability state is shared across modules and can easily become inconsistent.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

test_race_management.py - TestAssignCarToRace (race_management)

Case I-114:

integration/tests/test_race_management.py::TestAssignCarToRace::test_assign_car_valid_car

Scenario: Assign car valid car.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the race_management module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-115:

integration/tests/test_race_management.py::TestAssignCarToRace::test_assign_car_unavailable_fails

Scenario: Assign car unavailable fails.

Why Needed: This case is needed because availability state is shared across modules and can easily become inconsistent.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-116:

integration/tests/test_race_management.py::TestAssignCarToRace::test_assign_car_damaged_fails

Scenario: Assign car damaged fails.

Why Needed: This case is needed because vehicle-condition rules affect how race_management interacts with race and mission flows.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-117:

integration/tests/test_race_management.py::TestAssignCarToRace::test_assign_car_nonexistent_fails

Scenario: Assign car nonexistent fails.

Why Needed: This case is needed because race_management must reject stale or invalid identifiers cleanly.

Expected Result: The operation should fail with KeyError because the referenced record does not exist.

Observed Result: Passed locally in the audited integration suite.

Case I-118:

integration/tests/test_race_management.py::TestAssignCarToRace::test_assign_car_sets_unavailable

Scenario: Assign car sets unavailable.

Why Needed: This case is needed because availability state is shared across modules and can easily become inconsistent.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

test_race_management.py - TestStartRace (race_management)

Case I-119:

integration/tests/test_race_management.py::TestStartRace::test_start_race_without_driver_fails

Scenario: Start race without driver fails.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the race_management module.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-120: integration/tests/test_race_management.py::TestStartRace::test_start_race_without_car_fails

Scenario: Start race without car fails.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

race_management module.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-121:

integration/tests/test_race_management.py::TestStartRace::test_start_race_with_both_succeeds

Scenario: Start race with both succeeds.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

race_management module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

test_race_management.py - TestGetRace (race_management)

Case I-122: integration/tests/test_race_management.py::TestGetRace::test_get_race_valid_id

Scenario: Get race valid id.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

race_management module.

Expected Result: The lookup should return the requested record set in a valid structure.

Observed Result: Passed locally in the audited integration suite.

Case I-123:

integration/tests/test_race_management.py::TestGetRace::test_get_race_invalid_id_raises_keyerror

Scenario: Get race invalid id raises keyerror.

Why Needed: This case is needed because race_management must reject stale or invalid identifiers cleanly.

Expected Result: The operation should fail with KeyError because the referenced record does not exist.

Observed Result: Passed locally in the audited integration suite.

test_race_management.py - TestListRaces (race_management)

Case I-124: integration/tests/test_race_management.py::TestListRaces::test_list_races_empty

Scenario: List races empty.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the race_management module.

Expected Result: The empty-state query should return an empty but well-formed result.

Observed Result: Passed locally in the audited integration suite.

Case I-125: integration/tests/test_race_management.py::TestListRaces::test_list_races_returns_all

Scenario: List races returns all.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the race_management module.

Expected Result: The lookup should return the requested record set in a valid structure.

Observed Result: Passed locally in the audited integration suite.

Case I-126: integration/tests/test_race_management.py::TestListRaces::test_list_races_filter_by_status

Scenario: List races filter by status.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the race_management module.

Expected Result: The query should return only the matching records in the correct order.

Observed Result: Passed locally in the audited integration suite.

test_registration.py - TestRegisterMember (registration)

Case I-127: integration/tests/test_registration.py::TestRegisterMember::test_register_valid_driver

Scenario: Register valid driver.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the registration module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-128: integration/tests/test_registration.py::TestRegisterMember::test_register_all_valid_roles

Scenario: Register all valid roles.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the registration module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-129: integration/tests/test_registration.py::TestRegisterMember::test_register_name_too_short

Scenario: Register name too short.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the registration module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-130:

integration/tests/test_registration.py::TestRegisterMember::test_register_name_two_chars_valid

Scenario: Register name two chars valid.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the registration module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-131:

integration/tests/test_registration.py::TestRegisterMember::test_register_name_fifty_chars_valid

Scenario: Register name fifty chars valid.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the registration module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-132: integration/tests/test_registration.py::TestRegisterMember::test_register_name_too_long

Scenario: Register name too long.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the registration module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-133: integration/tests/test_registration.py::TestRegisterMember::test_register_invalid_role

Scenario: Register invalid role.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the registration module.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-134:

integration/tests/test_registration.py::TestRegisterMember::test_register_same_name_twice_allowed

Scenario: Register same name twice allowed.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the registration module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

test_registration.py - TestGetMember (registration)

Case I-135: integration/tests/test_registration.py::TestGetMember::test_get_member_valid_id

Scenario: Get member valid id.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the registration module.

Expected Result: The lookup should return the requested record set in a valid structure.

Observed Result: Passed locally in the audited integration suite.

Case I-136:

integration/tests/test_registration.py::TestGetMember::test_get_member_invalid_id_raises_keyerror

Scenario: Get member invalid id raises keyerror.

Why Needed: This case is needed because registration must reject stale or invalid identifiers cleanly.

Expected Result: The operation should fail with KeyError because the referenced record does not exist.

Observed Result: Passed locally in the audited integration suite.

test_registration.py - TestListMembers (registration)

Case I-137: integration/tests/test_registration.py::TestListMembers::test_list_members_empty

Scenario: List members empty.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the registration module.

Expected Result: The empty-state query should return an empty but well-formed result.

Observed Result: Passed locally in the audited integration suite.

Case I-138: integration/tests/test_registration.py::TestListMembers::test_list_members_returns_all

Scenario: List members returns all.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the registration module.

Expected Result: The lookup should return the requested record set in a valid structure.

Observed Result: Passed locally in the audited integration suite.

test_registration.py - TestRemoveMember (registration)

Case I-139: integration/tests/test_registration.py::TestRemoveMember::test_remove_member_valid_id

Scenario: Remove member valid id.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the registration module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-140:

integration/tests/test_registration.py::TestRemoveMember::test_remove_member_invalid_id_raises_keyerro

r

Scenario: Remove member invalid id raises keyerror.

Why Needed: This case is needed because registration must reject stale or invalid identifiers cleanly.

Expected Result: The operation should fail with KeyError because the referenced record does not exist.

Observed Result: Passed locally in the audited integration suite.

Case I-141:

integration/tests/test_registration.py::TestRemoveMember::test_remove_member_makes_unavailable

Scenario: Remove member makes unavailable.

Why Needed: This case is needed because availability state is shared across modules and can easily

become inconsistent.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

test_results.py - TestRecordResult (results)

Case I-142:

integration/tests/test_results.py::TestRecordResult::test_record_result_valid_in_progress_race

Scenario: Record result valid in progress race.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the results module.

Expected Result: The operation should succeed and leave the shared system state consistent.

Observed Result: Passed locally in the audited integration suite.

Case I-143: integration/tests/test_results.py::TestRecordResult::test_record_result_upcoming_race_fails

Scenario: Record result upcoming race fails.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the results module.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-144: integration/tests/test_results.py::TestRecordResult::test_record_result_wrong_winner_fails

Scenario: Record result wrong winner fails.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the results module.

Expected Result: The operation should be rejected and raise or return the documented validation failure.

Observed Result: Passed locally in the audited integration suite.

Case I-145:

integration/tests/test_results.py::TestRecordResult::test_record_result_updates_cash_balance

Scenario: Record result updates cash balance.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the results module.

Expected Result: The workflow should update shared cash state by the expected amount.

Observed Result: Passed locally in the audited integration suite.

Case I-146: integration/tests/test_results.py::TestRecordResult::test_record_result_updates_car_condition

Scenario: Record result updates car condition.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the results module.

Expected Result: The workflow should update the car condition field to match the race result.

Observed Result: Passed locally in the audited integration suite.

Case I-147: integration/tests/test_results.py::TestRecordResult::test_record_result_frees_driver

Scenario: Record result frees driver.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the results module.

Expected Result: Completing the workflow should release the reserved resource back into availability.

Observed Result: Passed locally in the audited integration suite.

Case I-148: integration/tests/test_results.py::TestRecordResult::test_record_result_frees_car

Scenario: Record result frees car.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the results module.

Expected Result: Completing the workflow should release the reserved resource back into availability.

Observed Result: Passed locally in the audited integration suite.

test_results.py - TestGetRankings (results)

Case I-149: integration/tests/test_results.py::TestGetRankings::test_get_rankings_empty

Scenario: Get rankings empty.

Why Needed: This case is needed because downstream views depend on results returning correctly ordered data.

Expected Result: The empty-state query should return an empty but well-formed result.

Observed Result: Passed locally in the audited integration suite.

Case I-150: integration/tests/test_results.py::TestGetRankings::test_get_rankings_single_winner

Scenario: Get rankings single winner.

Why Needed: This case is needed because downstream views depend on results returning correctly ordered data.

Expected Result: The lookup should return the requested record set in a valid structure.

Observed Result: Passed locally in the audited integration suite.

Case I-151: integration/tests/test_results.py::TestGetRankings::test_get_rankings_sorted_by_wins

Scenario: Get rankings sorted by wins.

Why Needed: This case is needed because downstream views depend on results returning correctly

ordered data.

Expected Result: The query should return only the matching records in the correct order.

Observed Result: Passed locally in the audited integration suite.

Case I-152:

integration/tests/test_results.py::TestGetRankings::test_get_rankings_sorted_by_earnings_secondary

Scenario: Get rankings sorted by earnings secondary.

Why Needed: This case is needed because downstream views depend on results returning correctly

ordered data.

Expected Result: The query should return only the matching records in the correct order.

Observed Result: Passed locally in the audited integration suite.

test_results.py - TestGetDriverStats (results)

Case I-153: integration/tests/test_results.py::TestGetDriverStats::test_get_driver_stats_after_race

Scenario: Get driver stats after race.

Why Needed: This case is needed because it verifies one of the core business rules implemented by the

results module.

Expected Result: The lookup should return the requested record set in a valid structure.

Observed Result: Passed locally in the audited integration suite.

Case I-154:

integration/tests/test_results.py::TestGetDriverStats::test_get_driver_stats_nonexistent_driver_fails

Scenario: Get driver stats nonexistent driver fails.

Why Needed: This case is needed because results must reject stale or invalid identifiers cleanly.

Expected Result: The operation should fail with KeyError because the referenced record does not exist.

Observed Result: Passed locally in the audited integration suite.

4. Diagram Submission Note

The final submission still needs the hand-drawn call graph image in `integration/diagrams/`. The Mermaid helper remains only a drawing reference.